==========

CO1_ASSESSMENT TOOL 1

APPLICATION - BASED QUESTION_2Q

==========

Submitted in partial fulfilment for the course of

CSA0606-Design and Analysis of Algorithm

for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

By,

Aadhithya.V(192511256)

UNDER THE GUIDANCE OF

Dr. Rajagopal.K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July 2026

Questions:

Q3. Application: E-Commerce Recommendation System An e-commerce system uses two algorithms with complexities O(n log n) and O(n2). Analyze both using asymptotic notation. Compare their performance for large datasets. Explain growth rate differences and justify which is more efficient. Provide a conclusion based on scalability.

Q43. Application: AI Model Training Pipeline An AI training system processes datasets using the recurrence T(n) = 9T(n/3) + n2. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze how training time grows with dataset size. Interpret efficiency using asymptotic notation.

Question 3

Application: E-Commerce Recommendation System

Aim

To analyze and compare the asymptotic time complexities of two recommendation algorithms having complexities **O(n log n)** and **O(n²)** and determine which algorithm is more efficient for large datasets.

Theory

An e-commerce platform recommends products to users based on browsing history, purchases, and preferences. Different recommendation algorithms require different execution times depending on the number of users or products.

Two algorithms are considered:

- Algorithm A: **O(n log n)**
- Algorithm B: **O(n²)**

Asymptotic notation helps compare algorithms independently of hardware and programming language.

Asymptotic Analysis

Algorithm A: O(n log n)

Time Complexity:

$$T(n) = O(nlogn)$$

Characteristics:

- Nearly linear growth.
- Performs efficiently even for large datasets.
- Commonly used in sorting and divide-and-conquer algorithms.

Algorithm B: O(n²)

Time Complexity:

$$T(n) = O(n^2)$$

Characteristics:

- Quadratic growth.
- Execution time increases rapidly.
- Suitable only for small datasets.

Growth Rate Comparison

Dataset Size (n)	O(n log n)	O(n²)
100	664	10,000
1,000	9,966	1,000,000
10,000	132,877	100,000,000
100,000	1,660,964	10,000,000,000

Comparison

O(n log n)

Advantages

- Faster execution.
- Better scalability.
- Efficient memory usage.

- Suitable for large e-commerce databases.

Disadvantages

- Slightly more complex implementation.

O(n²)

Advantages

- Simple implementation.
- Suitable for small datasets.

Disadvantages

- Very slow for large datasets.
- Poor scalability.

Performance Analysis

As the number of products increases,

- O(n log n) grows slowly.
- O(n²) grows much faster.

For example,

For n = 100000

- O(n log n) ≈ 1.66 million operations
- O(n²) = 10 billion operations

Hence, the quadratic algorithm becomes impractical.

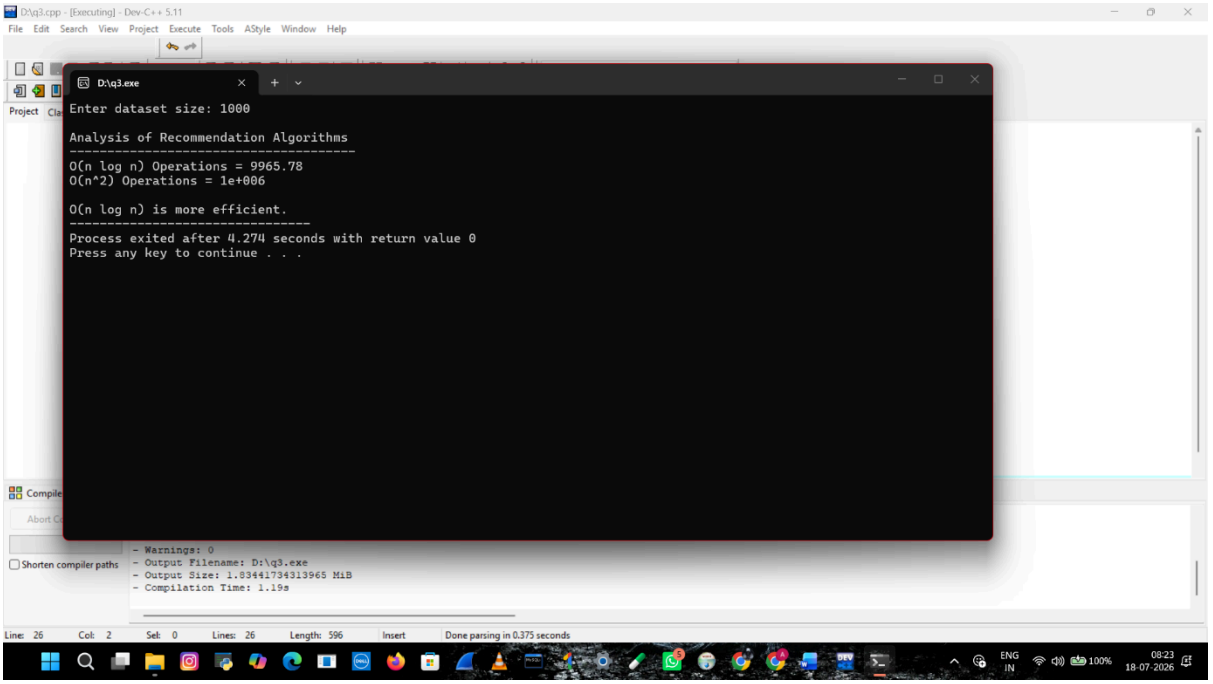
Conclusion

Algorithm **O(n log n)** is significantly more efficient than **O(n²)** for large datasets. It provides better scalability, lower execution time, and is more suitable for modern e-commerce recommendation systems handling millions of products and users.

Source Code:

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
   int n;

   cout << "Enter dataset size: ";
   cin >> n;

   double nlogn = n * log2(n);
   double nsquare = n * n;

   cout << "\nAnalysis of Recommendation Algorithms\n";
   cout << "-------------------------------------\n";
   cout << "O(n log n) Operations = " << nlogn << endl;
   cout << "O(n^2) Operations = " << nsquare << endl;

   if(nlogn < nsquare)
      cout << "\nO(n log n) is more efficient.";
   else
      cout << "\nO(n^2) is more efficient.";

   return 0;
}
```

Sample Output:



Question 43

Application: AI Model Training Pipeline

Aim

To determine the time complexity of the recurrence

$$T(n) = 9T(n/3) + n^2$$

using the Master Theorem and analyze the scalability of AI model training.

Theory

AI model training often divides datasets into smaller subsets recursively.

The recurrence is

$$T(n) = 9T(n/3) + n^2$$

where

- Recursive calls = 9
- Problem size reduces by 3
- Additional work = n²

The Master Theorem is used to determine the overall complexity.

Step 1: Identify Parameters

Given

$$T(n) = aT(n/b) + f(n)$$

Parameters are

- a = 9
- b = 3
- f(n)=n²

Step 2: Compute

$$n^{log_b a} = n^{log_3 9}$$

Since

$$log_3 9 = 2$$

Therefore

$$n^{log_b a} = n^2$$

Step 3: Compare

$$f(n) = n^2 n^{log_b a} = n^2$$

Both are equal.

Step 4: Apply Master Theorem

Since

$$f(n) = \Theta(n^{log_b a})$$

This satisfies Case 2 of the Master Theorem.

Therefore,

$$T(n) = \Theta(n^2 logn)$$

Time Complexity

$$\Theta(n^2 logn)$$

Growth Analysis

Dataset doubles:

Training time increases approximately by

$$n^2 logn$$

The quadratic component dominates while the logarithmic factor adds moderate growth.

Efficiency Analysis

Advantages

- Better than cubic algorithms.
- Suitable for divide-and-conquer AI models.
- Efficient for moderate to large datasets.

Disadvantages

- Training time still increases significantly for extremely large datasets.

Conclusion

Applying the Master Theorem,

$$T(n) = 9T(n/3) + n^2$$

results in

$$\Theta(n^2 logn)$$

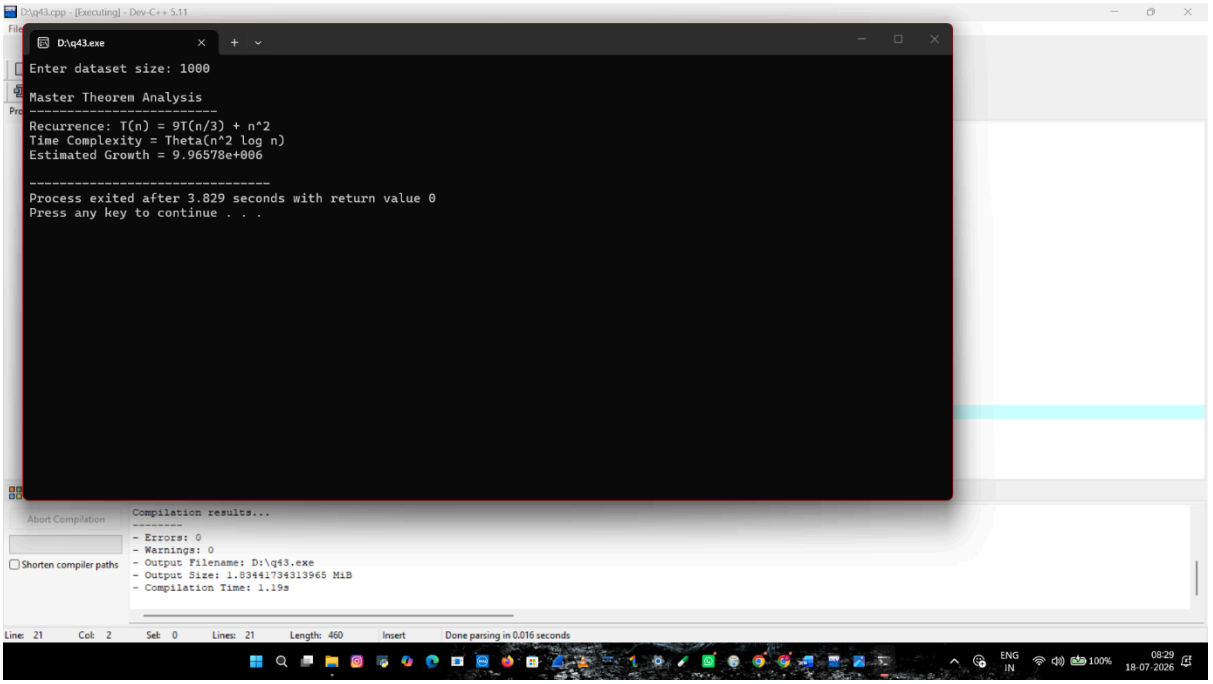
This indicates that AI model training grows faster than quadratic time but slower than cubic time. The divide-and-conquer strategy improves scalability

compared with many higher-order algorithms, making it suitable for large-scale AI applications.

Source Code:

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
   int n;

   cout << "Enter dataset size: ";
   cin >> n;

   double complexity = n * n * log2(n);

   cout << "\nMaster Theorem Analysis\n";
   cout << "-------------------------\n";
   cout << "Recurrence: T(n) = 9T(n/3) + n^2\n";
   cout << "Time Complexity = Theta(n^2 log n)\n";
   cout << "Estimated Growth = " << complexity << endl;

   return 0;
}
```

Sample Output:


D:\q43.exe
Enter dataset size: 1000
Master Theorem Analysis
Recurrence: T(n) = 9T(n/3) + n^2
Time Complexity = Theta(n^2 log n)
Estimated Growth = 9.96578e+006
Process exited after 3.829 seconds with return value 0
Press any key to continue . . .
Abort Compilation
Shorten compiler paths
Compilation results...
- Errors: 0
- Warnings: 0
- Output Filename: D:\q43.exe
- Output Size: 1.83441734313965 MiB
- Compilation Time: 1.19s
Done parsing in 0.016 seconds